

CHAPTER THREE

PROGRAMMING WITH 8086 MICROPROCESSOR

BY

Suroj Burlakoti

Lecturer

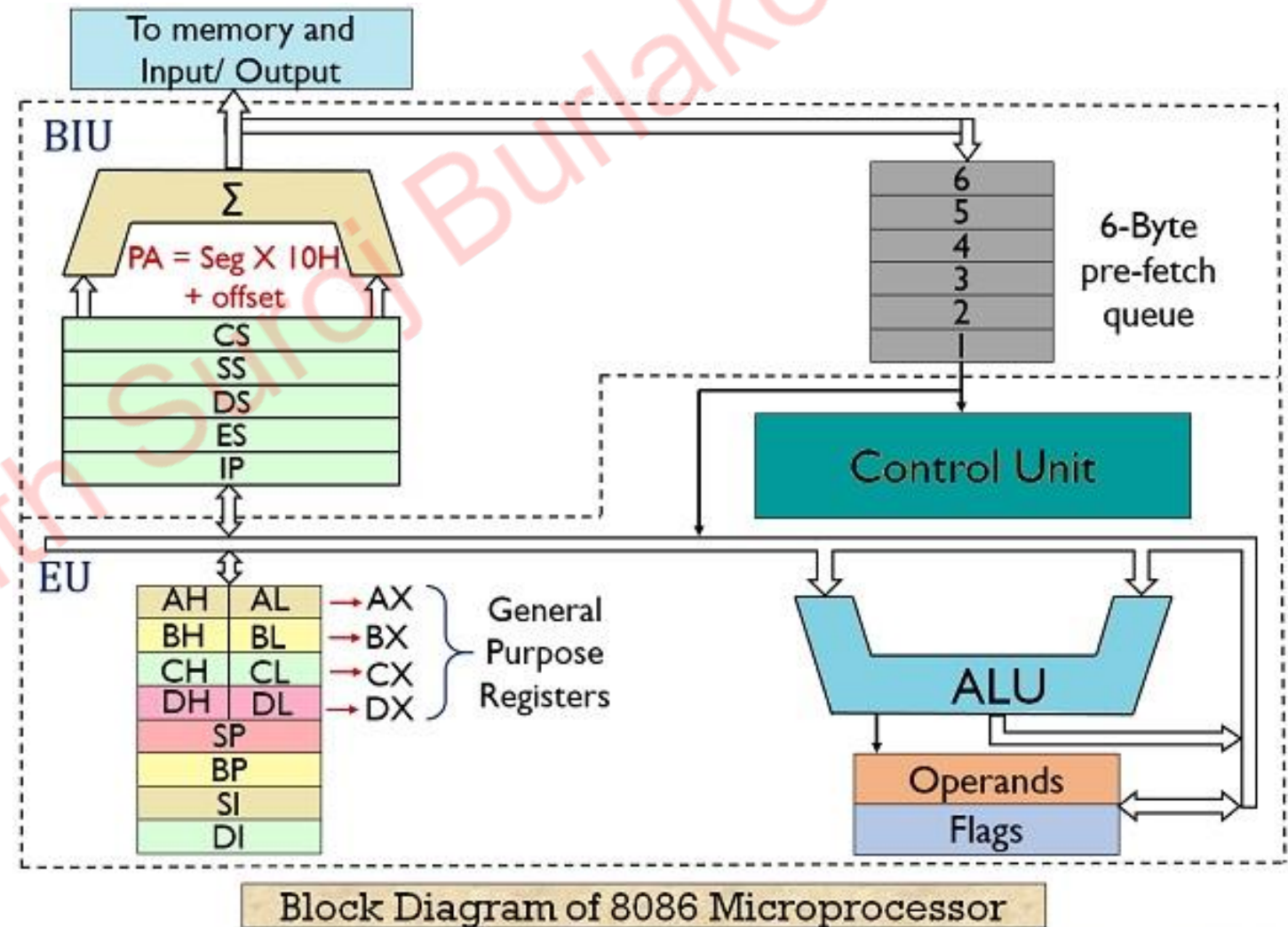
DoECE, National College of Engineering

Features of 8086 Microprocessor

- 16-bit microprocessor
- Can access 16-bit words from/to memory
- Has 20-bit address bus
- Uses a segmented memory
- Memory is organized in bytes, each byte can be addressed with 20-bit address of 00000- FFFFFH
- 8086 words consist of two consecutive bytes: A lower byte in the lower address and a higher byte in the higher address.
- Pre-fetches up to six instructions byte from memory and queues them in order to speed up the execution of the instructions
- Register based microprocessor

Internal Architecture of 8086 Microprocessor

- Internally divided into two logical unit
 - Bus Interface Unit (BIU)
 - Execution Unit (EU)
- BIU Fetches instruction, reads operands and write results
- EU executes instructions that have been fetched by BIU



Bus Interface Unit (BIU)

- BIU contains segment registers, instruction queue, bus control unit to provide functions such as fetching and queuing.
- It manages the above units and also control the transfer of data to EU, memory and external I/O devices.
- Segment registers control memory addressing
- BIU places the instruction in queue called Instruction Queue which is FIFO group of registers in which up to six byte of instruction codes are pre-fetched from memory ahead of time so that there is always queue of instructions ready to execute
- This mechanism is called pipelining which speeds up processing
- BIU contains a dedicated address generation control units to produce 20 bit address
- The bus control logic generates necessary control signals like read, write for memory and I/O

Segments

- Segments are special areas defined in a program for containing code, data and stack
- It begins on paragraph boundary i.e. at location exactly divisible by sixteen

1. Code Segment (CS)

- Code segment contains instructions that are to be executed
- CS register addresses the starting address of code segment

2. Data Segment (DS)

- It contains program defined data, constants etc.
- DS register addresses the starting address of data segment

Segments

3. Stack Segment (SS)

- Stack segment contains any data and addresses that the program needs to save temporarily or for subroutines
- SS register addresses the starting address of Stack Segment

4. Extra Segment (ES)

- It contains the excess data or program not accommodate in previous segments
- ES register addresses the starting address of Extra Segment

5. Instruction Pointer (IP)

- It contains the distance or offset from the address of CS to the next instruction byte to be fetched

Offset

- The distance in bytes from segment address to another location within the segment is called offset

a) CS & IP

- CS contains the starting address of a program code segment and IP contains the offset address of the particular instructions.
- The content of IP indicates the exact address of any instructions
- BIU computes 20 bit physical address internally using the contents of CS and IP
- First 16 bit content of CS is shifted four bit to left and then 16 bit content of IP is added to the shifted value to obtain the 20 bit exact physical address
- E.g. CS= 8000 , IP = 0012 $\rightarrow$ $80000 + 0012 = 80012 \leftarrow$ PA (Physical Address)

Offset

b) DS & BP

- DS register puts starting address of a program's data and the Base Pointer (BP) contains the offset address of specific byte location in data segment

c) SS and SP

- SS register points to starting address of a program's stack segment and Stack Pointer (SP) contains the offset address in the segment which finally indicates the exact address of stack being addressed

Offset

d) ES & DI or SI

- ES points to the starting address of extra segment and it is used in combination of Source Index (SI) or Destination Index (DI) for memory addressing in string operations

CS: IP

DS:BP DS:SI DS:DI DS:BX DS: Address

SS: BP SS: SP

ES: SI ES:DI

Execution Unit (EU)

- It decodes and executes instructions that have been fetched by the BIU.
- it consist of ALU (16 bit), Control Unit (CU), and the no. of registers
- It maintain microprocessor starter, control flags, manipulates general purpose registers and instruction operands.
- It has nine 16 bit registers: AX, BX, CX, DX, SP, BP, SI, DI and Flag register
- Some 16 bit register (AX, BX, CX and DX) can also be used as individual 8-bit register (AL,AH,BL,BH,CL,CH,DL,DH)

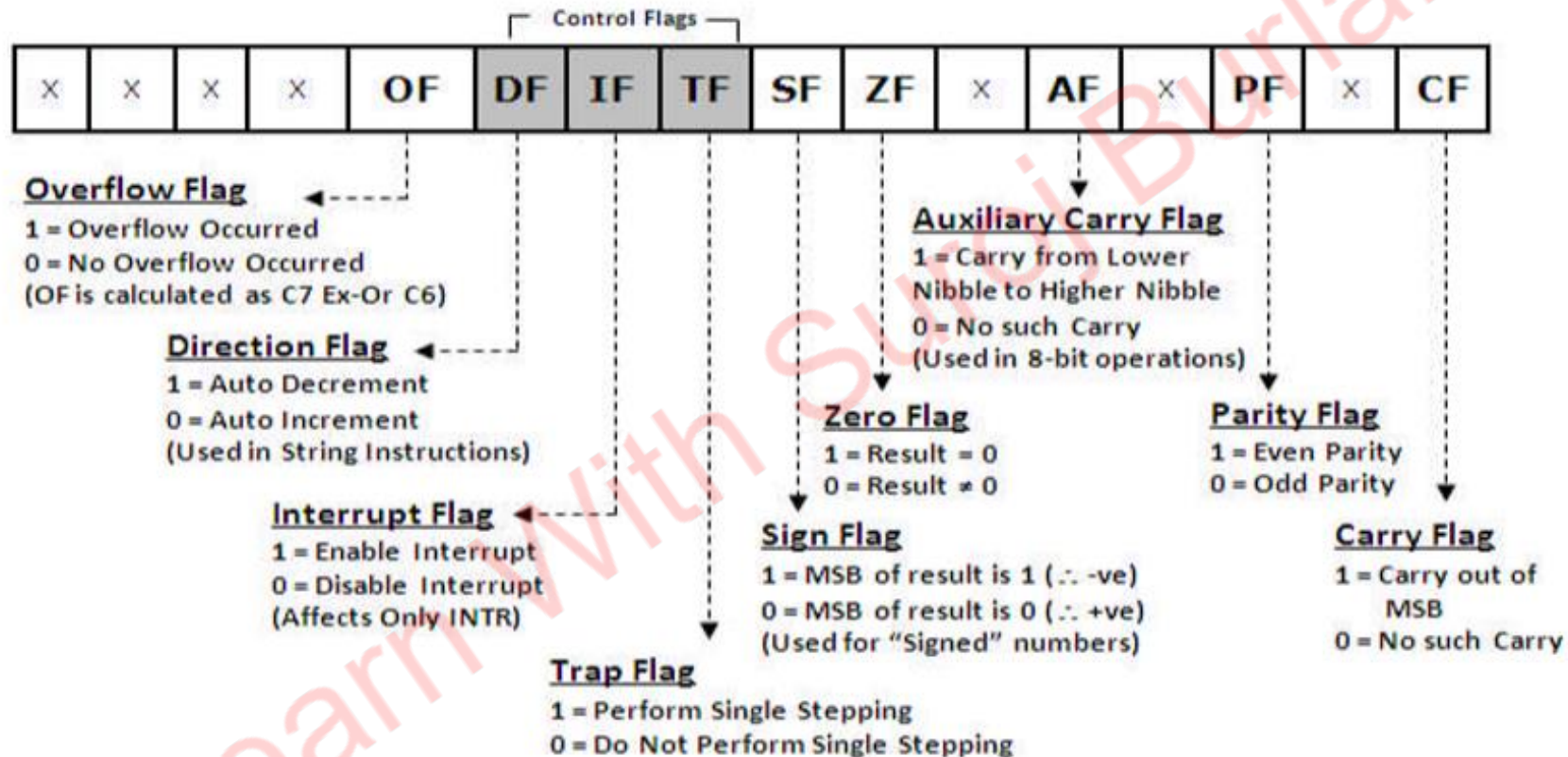
Registers

- **AX (Accumulator):** IN and OUT instructions use accumulator for input and output and arithmetic operations
- **BX (Base Register):** Used also for addressing besides computation
- **CX (Count Register):** Used for looping and computation purpose
- **DX (Data Register):** For a normal computation, especially for multiplication and division pairing with AX

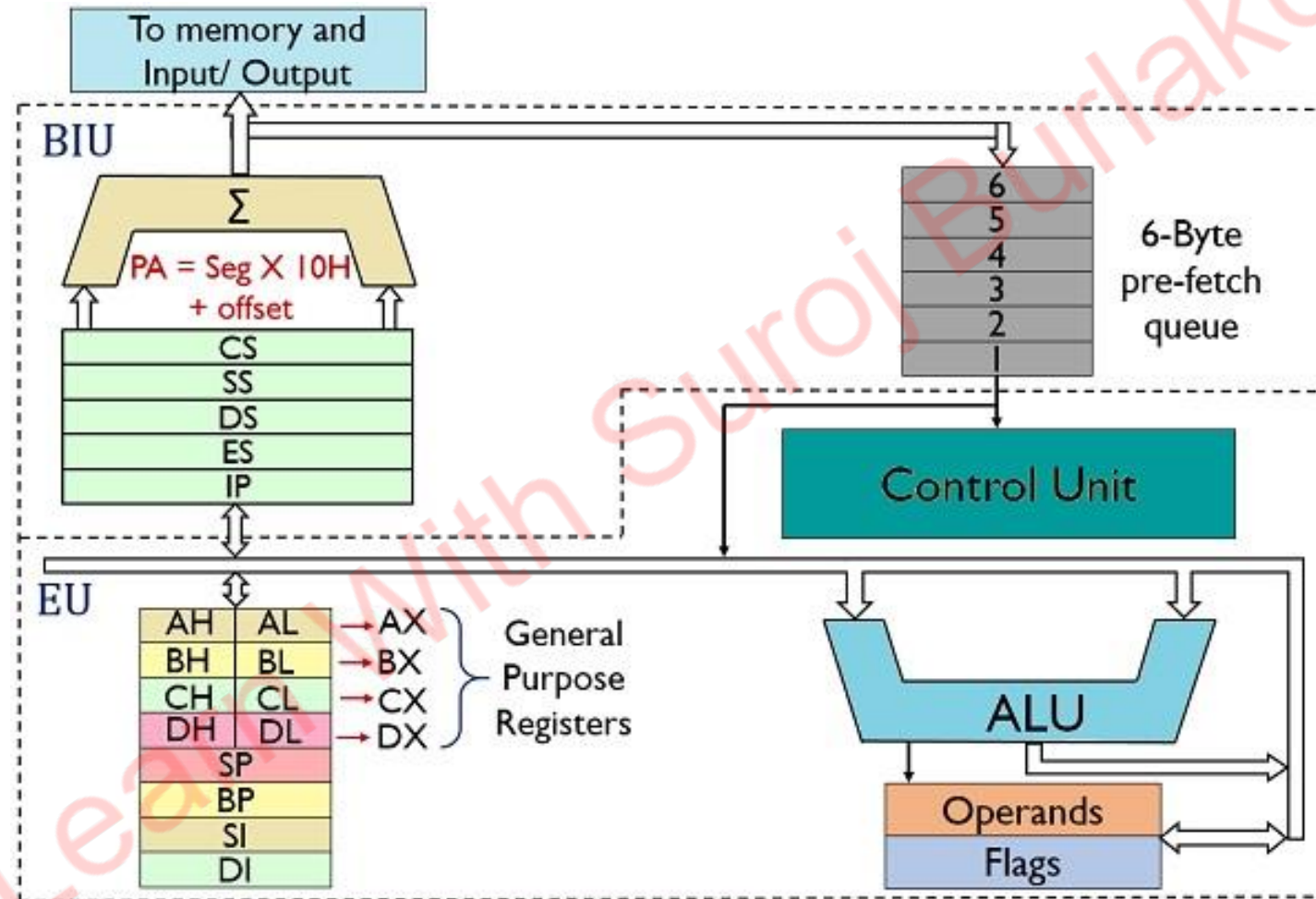
Registers

- **SP (Stack Pointer):**
 - It is used as offset from current stack segment during stack operation
 - It's content are automatically incremented or decremented during POP and PUSH instructions
- **BP (Base Pointer):**
 - Contains an offset address to reference data in data segment or stack segment
- **Index Register**
 - **SI (Source Index):** Used for some string operation in combination with DS or ES segment registers
 - **DI (Destination Index):** Also used for string operations in combination with ES

Flag Registers



Internal Architecture of 8086 Microprocessor



Block Diagram of 8086 Microprocessor

Instructions in 8086

1) Arithmetic Instructions

a) **ADD** $\text{reg}_8/\text{mem}_8, \text{reg}_8/\text{mem}_8/\text{Immediate}_8$

ADD $\text{reg}_{16}/\text{mem}_{16}, \text{reg}_{16}/\text{mem}_{16}/\text{Immediate}_{16}$

E.g. ADD AH, 15 ; It adds binary number

ADD AH, NUM1 ADD AI, [BX]

ADD [BX], CH/CX ADD AX,[BX]

b) **ADC**: Addition with Carry

ADC $\text{reg}/\text{mem}, \text{reg}/\text{mem}/\text{Immediate data}$

c) **SUB**: Subtract 8 bit or 16 bit binary numbers

SUB $\text{reg}/\text{mem}, \text{reg}/\text{mem}/\text{Immediate}$

Instructions in 8086

d) **SBB**: Subtract with borrow

SBB reg/mem, reg/mem/Immediate

e) **MUL** : unsigned multiplication

MUL reg₈/mem₈ (8 bit accumulator – AL)

MUL reg₁₆/ mem₁₆ (16 bit accumulator-Ax)

E.g. MUL R₈ (*multiplier*) $\equiv$ R₈ × AL → AX (16 bit result)

MUL R₁₆ (*multiplier*) $\equiv$ R₁₆ × AL → DX:AX (32 bit result)

IMUL – signed multiplication

Same operation as MUL but takes sign into account

Instructions in 8086

f) **DIV** reg/mem

E.g. $\text{DIV } R_8 \equiv \text{AX}/R_8 \rightarrow (\text{Remainder} \rightarrow \text{AH}) \ \& \ (\text{Quotient} \rightarrow \text{AL})$

$\text{DIV } R_{16} \equiv \text{DX:AX}/R_{16} \rightarrow (\text{Remainder} \rightarrow \text{DX}) \ \& \ (\text{Quotient} \rightarrow \text{AX})$

IDIV- Signed division

Same operation as DIV but takes sign into account.

g) **INC/DEC** (Increment/Decrement by 1)

INC/DEC reg./mem. (8 bit or 16bit)

E.g. `INC AL` `DEC BX`

`INC NUM1`

Instructions in 8086

- h) **NEG**- Negate (2's complement)
- i) **ASCII-BCD** Conversion
 - AAA: ASCII Adjust after addition
 - AAS: ASCII Adjust after subtraction
 - AAM: Adjust after multiplication
 - AAD: Adjust after division
 - DAA: Decimal adjust after addition
 - DAS: Decimal adjust after subtraction

Instructions in 8086

2) Logical/shifting/comparison instructions

a) Logical

AND/OR/XOR reg/mem, reg/mem/immediate

NOT reg/mem

E. g. AND AL, AH

XOR [BX], CL

b) Rotation

ROL- rotate left, ROR-rotate right

E.g. ROL AX, 1 ; rotated by 1

ROL AX, CL ; if we need to rotate more than one bit

RCL-rotate left through carry

RCR-rotate right through carry

E.g. RCL AX, 1

RCL AX, CL ; Only CL can be used

Instructions in 8086

c) Shifting

SHL -logical shift left

SHR - logical shift right

Shifts bit in true direction and fills zero in vacant place

E.g. SHL reg/mem, 1/CL

arithmetic shift left

SAR- arithmetic shift right

Shifts bit/word in true direction, in former case place zero in vacant place and in later case place previous sign in vacant place.

E.g. 1 011010 [1 $\Rightarrow$ 11011010

Instructions in 8086

d) Comparison

CMP –compare

CMP reg/mem, reg/mem/immediate

E.g. CMP BH, AL

Operand1		Operand 2	CF	SF	ZF
	>		0	0	0
	=		0	0	1
	<		1	1	0

TEST: test bits (AND operation)

TEST reg/mem, reg/mem/immediate

Instructions in 8086

3) Data Transfer Instructions:

MOV reg./mem , reg./mem./immediate

LDS: Load data segment register

LEA: load effective address

LES: Load extra segment register

LSS: Load stack segment register

E.g. LEA BX, ARR = MOV BX, OFFSET ARR

LDS BX, NUM1

Segment address → DS

Offset address → BX

XCHG reg/mem, reg/mem

E.g. XCHG AX, BX

XCHG AL, BL

XCHG CL,[BX]

IN AL, DX ; DX: Port address, AH also in AL

OUT DX, AL/AH

Instructions in 8086

4) Flag Operation

CLC: Clear carry flag

CLD: Clear direction flag

CLI: Clear interrupt flag

STC: Set Carry flag

STD: Set direction flag

STI: Set Interrupt flag

CMC: Complement Carry flag

LAHF: Load AH from flags (lower byte)

SAHF: Store AH to flags

PUSHF: Push flags into stack

POPF: Pop flags off stack

Instructions in 8086

5) STACK Operations

PUSH reg₁₆

POP reg₁₆

6) Looping instruction (CX is automatically used as a counter)

LOOP: loop until complete

LOOPE: Loop while equal

LOOPZ: loop while zero

LOOPNE: loop while not equal

LOOPNZ: loop while not zero

Instructions in 8086

7) Branching instruction

a) Conditional

JA: Jump if Above

JAE: Jump if above/equal

JB: Jump if below

JBE: Jump if below/equal

JC: Jump if carry

JNC: Jump if no carry

JE: Jump if equal

JNE: Jump if no equal

JZ: Jump if zero

JNZ: Jump if no zero

JG: Jump if greater

JNG: Jump if no greater

JL: Jump if less

JNL: Jump if no less

JO: jump if overflow

JS: Jump if sign

JNS: Jump if no sign

JP: jump if plus

JPE: Jump if parity even

JNP: Jump if no parity

JPO: Jump if parity odd

Instructions in 8086

b) Unconditional

CALL: call a procedure

RET: Return

INT: Interrupt

IRET: interrupt return

JMP: Unconditional Jump

RETN/RETF: Return near/Far

8) Type conversion

CBW: Convert byte to word

CWD: Convert word to double word

AX → DX: AX

Instructions in 8086

9) String instructions

- a) MOVS/ MOVSB/MOVSX ; Move string
DS: SI source
DS: DI destination
CX: String length
- b) CMPS/ CMPSB/CMPSW ; Compare string
- c) LODS /LODSB/LODSW ; Load string
- d) REP ; Repeat string

Addressing Modes of 8086

1) Register Addressing:

For this mode, a register may contain source operand, destination operand or both.

E.g. MOV AH, BL
 MOV DX, CX

2) Immediate Addressing

In this type of addressing, immediate data is a part of instruction, and appears in the form of successive byte or bytes. This mode contains a constant value or an expression.

E.g. MOV AH, 35H
 MOV BX, 7A25H

Addressing Modes of 8086

3) Direct memory addressing:

In this type of addressing mode, a 16-bit memory address (offset) is directly specified in the instruction as a part of it. One of the operand is the direct memory and other operand is the register.

E.g. `ADD AX, [5000H]`

Note: Here data resides in a memory location in the data segment, whose effective address may be computed using 5000H as the Offset address and content of DS as segment address. The effective address, here, is $10H * DS + 5000H$.

4) Direct offset addressing

In this addressing, a variation of direct addressing uses arithmetic operators to modify an address.

E.g. `ARR DB 15, 17, 18, 21`
`MOV AL, ARR [2] ; MOV AL, 18`
`ADD BH, ARR+3 ; ADD BH, 21`

Addressing Modes of 8086

5) Indirect memory addressing:

Indirect addressing takes advantage of computer's capability for segment: offset addressing. The registers used for this purpose are base register (BX and BP) and index register (DI and SI)

E.g. MOV [BX], AL
 ADD CX, [SI]

6) Base displacement addressing:

This addressing mode also uses base registers (BX and BP) and index register (SI and DI), but combined with a displacement (a number or offset value) to form an effective address.

E.g. MOV BX, OFFSET ARR
 $\equiv$ LEA BX, ARR
 MOV AL, [BX + 2]
 ADD TBL [BX], CL
 TBL [BX] $\longrightarrow$ [BX + TBL] e.g. [BX + 4]

Addressing Modes of 8086

7) Base index addressing:

This addressing mode combines a base registers (BX or BP) with an index register (SI or DI) to form an effective address.

E.g. MOV AX, [BX +SI]
 ADD [BX+DI], CL

8) Base index with displacement addressing

This addressing mode, a variation on base- index combines a base register, an index register, and a displacement to form an effective address.

E.g. MOV AL, [Bx+SI+2]
 ADD TBL [BX +SI], CH

Addressing Modes of 8086

9) String addressing:

This mode uses index registers, where SI is used to point to the first byte or word of the source string and DI is used to point to the first byte or word of the destination string, when string instruction is executed. The SI or DI is automatically incremented or decremented to point to the next byte or word depending on the direction flag (DF).

E.g. MOVS, MOVSB, MOVSW

Coding in Assembly language

- Assembly language programming language has taken its place in between the machine language (low level) and the high level language.
- High level language's one statement may generate many machine instructions.
- Low level language consists of either binary or hexadecimal operation. One symbolic statement generates one machine level instructions.
- **Advantage of ALP**
 - They generate small and compact execution module.
 - They have more control over hardware.
 - They generate executable module and run faster.
- **Disadvantages of ALP:**
 - Machine dependent.
 - Lengthy code
 - Error prone (likely to generate errors).

Assembly language features

- The main features of ALP are program comments, reserved words, identifies, statements and directives which provide the basic rules and framework for the language.

Program comments:

- The use of comments throughout a program can improve its clarity.
- It starts with semicolon (;) and terminates with a new line.
- E.g. `ADD AX, BX ; Adds AX & BX`

Reserved words:

- Certain names in assembly language are reserved for their own purpose to be used only under special conditions and includes
- Instructions : Such as MOV and ADD (operations to execute)
- Directives: Such as END, SEGMENT (information to assembler)
- Operators: Such as FAR, SIZE
- Predefined symbols: such as @DATA, @ MODEL

Identifiers:

- An identifier (or symbol) is a name that applies to an item in the program that expects to reference.
- Two types of identifiers are Name and Label.
- Name refers to the address of a data item such as NUM1 DB 5, COUNT DB 0
- Label refers to the address of an instruction.
- E. g: MAIN PROC FAR
- L1: ADD BL, 73

Statements:

- ALP consists of a set of statements with two types
- Instructions, e. g. MOV, ADD
- Directives, e. g. define a data item

	Identifiers	operation	operand	comment
Directive:	COUNT	DB	1	; initialize count
Instruction:	L30:	MOV	AX, 0	; assign AX with 0

Directives:

The directives are the number of statements that enables us to control the way in which the source program assembles and lists. These statements called directives act only during the assembly of program and generate no machine-executable code. The different types of directives are:

1) The page and title listing directives:

The page and title directives help to control the format of a listing of an assembled program. This is their only purpose and they have no effect on subsequent execution of the program.

The page directive defines the maximum number of lines to list as a page and the maximum number of characters as a line.

PAGE [Length] [Width]

Default: Page [50][80]

TITLE gives title and place the title on second line of each page of the program.

TITLE text [comment]

2) **SEGMENT directive**

It gives the start of a segment for stack, data and code.

Seg-name Segment [align][combine]['class']

Seg-name ENDS

- Segment name must be present, must be unique and must follow assembly language naming conventions.
- An ENDS statement indicates the end of the segment.
- Align option indicates the boundary on which the segment is to begin; PARA is used to align the segment on paragraph boundary.
- Combine option indicates whether to combine the segment with other segments when they are linked after assembly. STACK, COMMON, PUBLIC, etc are combine types.
- Class option is used to group related segments when linking. The class code for code segment, stack for stack segment and data for data segment.

3) PROC Directives

The code segment contains the executable code for a program, which consists of one or more procedures, defined initially with the PROC directives and ended with the ENDP directive.

PROC - name PROC [FAR/NEAR]

.....

.....

.....

PROC - name ENDP

- FAR is used for the first executing procedure and rest procedures call will be NEAR.
- Procedure should be within segment.

4) END Directive

- An END directive ends the entire program and appears as the last statement.
- ENDS directive ends a segment and ENDP directive ends a procedure. END PROC-Name

5) **ASSUME Directive**

- An .EXE program uses the SS register to address the stack, DS to address the data segment and CS to address the code segment.
- Used in conventional full segment directives only.
- Assume directive is used to tell the assembler the purpose of each segment in the program.
- Assume SS: Stack name, DS: Data Segname CS: codesegname

6) **Processor directive**

- Most assemblers assume that the source program is to run on a basic 8086 level computer.
- Processor directive is used to notify the assembler that the instructions or features introduced by the other processors are used in the program.

E.g. **.386** - program for 386 protected mode.

7) Dn Directive (Defining data types)

Assembly language has directives to define data syntax [name] Dn expression

The Dn directive can be any one of the following:

DB	Define byte	1 byte
DW	Define word	2 bytes
DD	Define double	4 bytes
DF	defined farword	6 bytes
DQ	Define quadword	8 bytes
DT	Define 10 bytes	10 bytes

VAL1 DB 25

ARR DB 21, 23, 27, 53

MOV AL, ARR [2] or

MOV AL, ARR + 2 ; Moves 27 to AL register

8) The EQU directive

- It can be used to assign a name to constants.
- E.g. **FACTOR EQU 12**
- MOV BX, FACTOR ; MOV BX, 12
- It is short form of equivalent.
- Do not generate any data storage; instead the assembler uses the defined value to substitute in.

9) DUP Directive

- It can be used to initialize several locations to zero.
e. g. SUM DW 4 DUP(0)
- Reserves four words starting at the offset sum in DS and initializes them to Zero.
- Also used to reserve several locations that need not be initialized. In this case (?) is used with DUP directives.
E. g. **PRICE DB 100 DUP(?)**
- Reserves 100 bytes of uninitialized data space to an offset PRICE.

Operators in 8086

- Operator can be applied in the operand which uses the immediate data/address.
- Being active during assembling and no machine language code is generated.
- Different types of operators are:

- 1) **Arithmetic**: + , - , * , /
- 2) **Logical** : AND, OR, XOR, NOT
- 3) **SHL and SHR**: Shift during assembly
- 4) **[]**: index
- 5) **HIGH**: returns higher byte of an expression
- 6) **LOW**: returns lower byte of an expression.
E.g. NUM EQU 1374 H
MOV AL HIGH Num ; ([AL] ← 13)
- 7) **OFFSET**: returns offset address of a variable
- 8) **SEG**: returns segment address of a variable
- 9) **PTR**: used with type specifications
BYTE, WORD, RWORD, DWORD, QWORD
E.g. INC BYTE PTR [BX]
- 10) **Segment override**
MOV AH, ES: [BX]

11) **LENGTH**: returns the size of the referred variable

12) **SIZE**: returns length times type

E.g.: **BYTE VAR DB?**

WTABLE DW 10 DUP (?)

MOV AX, TYPE BYTEVAR ; AX = 0001H

MOV AX, TYPE WTABLE ; AX = 0002H

MOV CX, LENGTH WTABLE ; CX = 000AH

MOV CX, SIZE WTABLE ; CX = 0014H

Writing a Program

Page 60, 132

TITLE Sum program to add two numbers.

.MODEL SMALL

.STACK 64

.DATA

NUM1 DW 3241

NUM 2 DW 572

SUM DW ?

.CODE

MAIN PROC FAR

MOV AX, @ DATA ; set address of data segment in DS

MOV DS, AX

MOV AX, NUM1

ADD AX, NUM2

MOV SUM, AX

MOV AX, 4C00H ; End processing

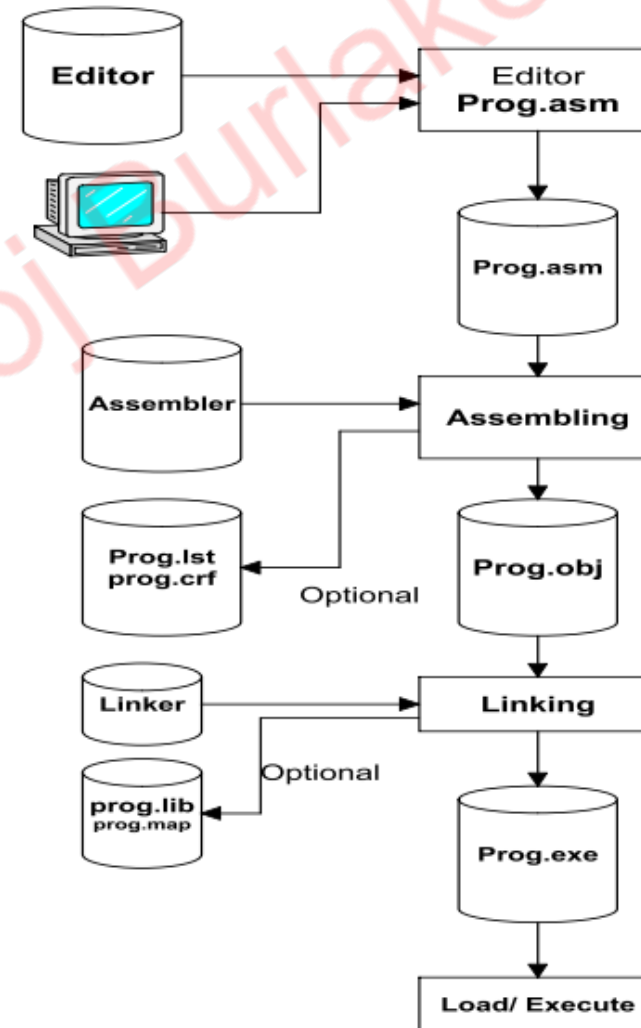
INT 21H

MAIN ENDP ; End of procedure

END MAIN ; End of program

Assembling, Linking and Executing

- The source program is just a text file (.txt) that cannot execute so we must assemble and link it



Assembling

- Assembling converts source program into object program if syntactically correct and generates an intermediate **.obj** file or module.
- It calculates the offset address for every data item in data segment and every instruction in code segment.
- A header is created which contains the incomplete address in front of the generated **obj** module during the assembling.
- Assembler complains about the syntax error if any and does not generate the object module.
- Assembler creates **.obj**, **.lst** and **.crf** files and last two are optional files that can be created at run time.
- For short programs, assembling can be done manually where the programmer translates each mnemonic into the machine language using lookup table.
- Assembler reads each assembly instruction of a program as ASCII character and translates them into respective machine code

Assembling

Assembler Types:

There are two types of assemblers:

a) One pass assembler:

- This assembler scans the assembly language program once and converts to object code at the same time.
- This assembler has the program of defining forward references only.
- The jump instruction uses an address that appears later in the program during scan, for that case the programmer defines such addresses after the program is assembled.

b) Two pass assembler

- This type of assembler scans the assembly language twice.
- First pass generates symbol table of names and labels used in the program and calculates their relative address.
- This table can be seen at the end of the list file and here user need not define anything.
- Second pass uses the table constructed in first pass and completes the object code creation.
- This assembler is more efficient and easier than earlier.

Linking

- This involves the converting of **.OBJ** module into **.EXE**(executable) module i.e. executable machine code.
- It completes the address left by the assembler.
- It combines separately assembled object files.
- Linking creates **.EXE**, **.LIB**, **.MAP** files among which last two are optional files.

Loading and Executing

- It Loads the program in memory for execution.
- It resolves remaining address.
- This process creates the program segment prefix (PSP) before loading.
- It executes to generate the result.

Sample program assembling → object Program linking → executable program

.EXE and .COM

	.COM Program	.EXE Program
1. Program Size	Single 64K-byte Segment	Multiple Segments 512 byte of header record that proceeds an .exe program
2. Segments	Only one segment for combining the PSP, Stack, Data, Code	Use different segment for defining data, stacking and coding purpose
3. Initialization	No need to initialization. Execution Always Begins At Offset 100h	Need to initialize
4. Model	Tiny	Small, Large, Huge, Compact

Macro- Assembler

- A macro is an instruction sequence that appears repeatedly in a program assigned with a specific name.
- The macro assembler replaces a macro name with the appropriate instruction sequence each time it encounters a macro name.
- When same instruction sequence is to be executed repeatedly, macro assemblers allow the macro name to be typed instead of all instructions provided the macro is defined.
- Macro are useful for the following purposes:
 - To simplify and reduce the amount of repetitive coding.
 - To reduce errors caused by repetitive coding.
 - To make an assembly language program more readable.
 - Macro executes faster because there is no need to call and return

Macro name	MACRO [Parameter list]	; Define macro
		
		
	[Instructions]	; Macro body
		
		
	ENDM	; End of macro

e.g.

Addition	MACRO
	IN AX, PORT
	ADD AX, BX
	OUT PORT, AX
	ENDM

DOS Function and Interrupts

[Keyboard and Video Services]

- The Intel CPU recognizes two types of interrupts namely hardware interrupt when a peripheral devices needs attention from the CPU and software interrupt that is call to a subroutine located in the operating system.
- The common software interrupts used here are **INT 10H for video services** and **INT 21H for DOS services**.

INT 21H [DOS Services]

- It is called the DOS function call for keyboard operations follow the function number. The service functions are listed below:
- **00H- It terminates the current program.**
 - Generally not used, function 4CH is used instead.
- **01H- Read a character with echo**
 - Wait for a character if buffer is empty
 - Character read is returned in AL in ASCII value
- **02H- Display single character**
 - Sends the characters in DL to display
- **03H and 04H – Auxiliary input/output**
 - INT 14H is preferred.
- **05H – Printer service**
 - Sends the character in DL to printer

- MOV AH, 02H
- MOV DL, 'A' ; move DL, 65
- INT 21H

INT 21H [DOS Services]

- **06H- Direct keyboard and display**
 - Displays the character in DL.
- **07H- waits for a character from standard input**
 - does not echo
- **08H- keyboard input without echo**
 - Same as function 01H but not echoed.
- **09H- string display**
 - Displays string until '\$' is reached.
 - DX should have the address of the string to be displayed.
- **0AH – Read string**
- **0BH- Check keyboard status**
 - Returns FF in AL if input character is available in keyboard buffer.
 - Returns 00 if not.
- **0CH- Clear keyboard buffer and invoke input functions such as 01, 06, 07, 08 or 0A.**
 - AL will contain the input function.

INT 21H Detailed for Useful Functions

01H

MOV, AH 01H; request keyboard input INT 21H

- Returns character in AL. IF AL= nonzero value, operation echoes on the screen. If AL= zero means that user has pressed an extended function key such as F1 OR home.

02H

MOV AH, 02H; request display character

MOV DL, CHAR; character to display

INT 21H

- Display character in D2 at current cursor position. The tab, carriage return and line feed characters act normally and the operation automatically advances the cursor.

09H

MOV Ah, 09H; request display

LEA DX, CUST_MSG; local address of prompt

INT 21H

CUST_MSG DB "Hello world", '\$'

- Displays string in the data area, immediately followed by a dollar sign (\$ or 24H), which uses to end the display.

0AH

MOV AH, 0AH ; request keyboard input

LEA DX, PARA_LIST ; load address of parameter list

INT 21H

INT 10H [BIOS Services]

- It is called video display control. It controls the screen format, color, text style, making windows, scrolling etc.

00H – set video mode

```
MOV AH, 00H      ; set mode
MOV AL, 03H      ; standard color text
INT 10H          ; call interrupt service
```

01H- set cursor size

```
MOV AH, 01H
MOV CH, 00H      ; Start scan line
MOV CL, 14H      ; End scan line
INT 10H          ; (Default size 13:14)
```

INT 10H [BIOS Services]

02H – Set cursor position:

```
MOV AH, 02H
MOV BH, 00H      ; page no
MOV DH, 12H      ; row/y (12)
MOV DL, 30H      ; column/x (30)
INT 10H
```

03H – return cursor status

```
MOV AH, 03H
MOV BH, 00H;
INT 10H
Returns: CH- starting scan line, CL-end scan line, DH- row, DL-column
```

04H- light pen function

05H- select active page

```
MOV AH, 05H
MOV AL,page-no.  ; page number
INT 10H
```

INT 10H [BIOS Services]

06H- scroll up screen

MOV AX, 060FH ; request scroll up one line (text)
MOV BH, 61H ; brown background, blue foreground
MOV CX, 0000H ; from 00:00 through
MOV DX, 184F H ; to 24:79 (full screen)

INT 10H

AL= number of rows (00 for full screen)

BH= Attribute or pixel value

CX= starting row: column

DX= ending row: column

07H-Scroll down screen

Same as 06H except for down scroll

08H (Read character and Attribute at cursor)

MOV AH, 08H

MOV BH, 00H ; page number 0(normal)

INT 10H

AL= character

BH= Attribute

INT 10H [BIOS Services]

09H -display character and attribute at cursor

```
MOV AH, 09H
MOV AL, 01H      ; ASCII for happy face display
MOV BH, 00H      ; page number
MOV BL, 16H      ; Blue background, brown foreground
MOV CX, 60       ; No of repeated character
INT 10H
```

0AH-display character at cursor

```
MOV AH, 0AH
MOV AL, Char
MOV BH, page_no
MOV BL, value
MOV CX, repetition
INT 10H
```

INT 10H [BIOS Services]

0BH- Set color palette

- ✓ Sets the color palette in graphics mode
- ✓ Value in BH (00 or 01) determines purpose of BL
- ✓ BH= 00H, select background color, BL contains 00 to 0FH (16 colors)
- ✓ BH = 01H , select palette, BL, contains palette

MOV AH, 0BH

MOV AH, 0BH

MOV BH, 00H; background

MOV BL, 04H; red

INT 21H

MOV BH, 01H ; select palette

MOV BL, 00H ; black

INT 21H

#0CH- write pixel Dot

- Display a selected color

AL=color of the pixel

CX= column

BH=page number

DX= row

MOV AH, 0CH

MOV AL, 03

MOV BH, 0

MOV CX, 200

MOV DX, 50

INT 10H

It sets pixel at column 200, row 50

INT 10H [BIOS Services]

#0DH- Read pixel dot

Reads a dot to determine its color value which returns in AL

```
MOV AH, 0DH
```

```
MOV BH, 0 ; page no
```

```
MOV CX, 80 ; column
```

```
MOV DX, 110 ; row
```

```
INT 10H
```

#0EH- Display in teletype mode

Use the monitor as a terminal for simple display

```
MOV AH, 0EH
```

```
MOV AL, char
```

```
MOV BL, color; foreground color
```

```
INT 10H
```

INT 10H [BIOS Services]

#0F H- Get current video mode

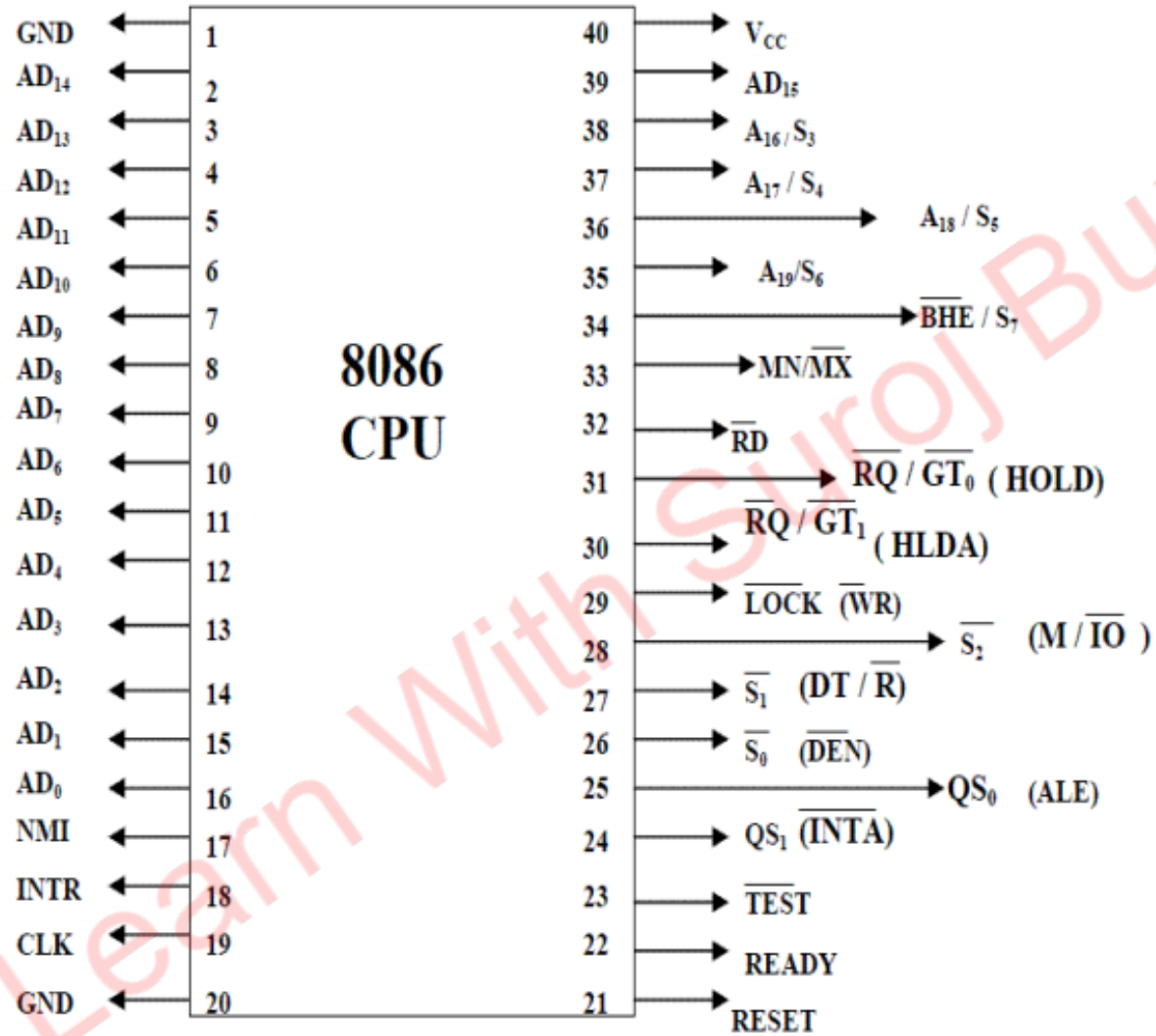
Returns values from the BIOS video .

AL= current video mode **MOV AH, 0FH**

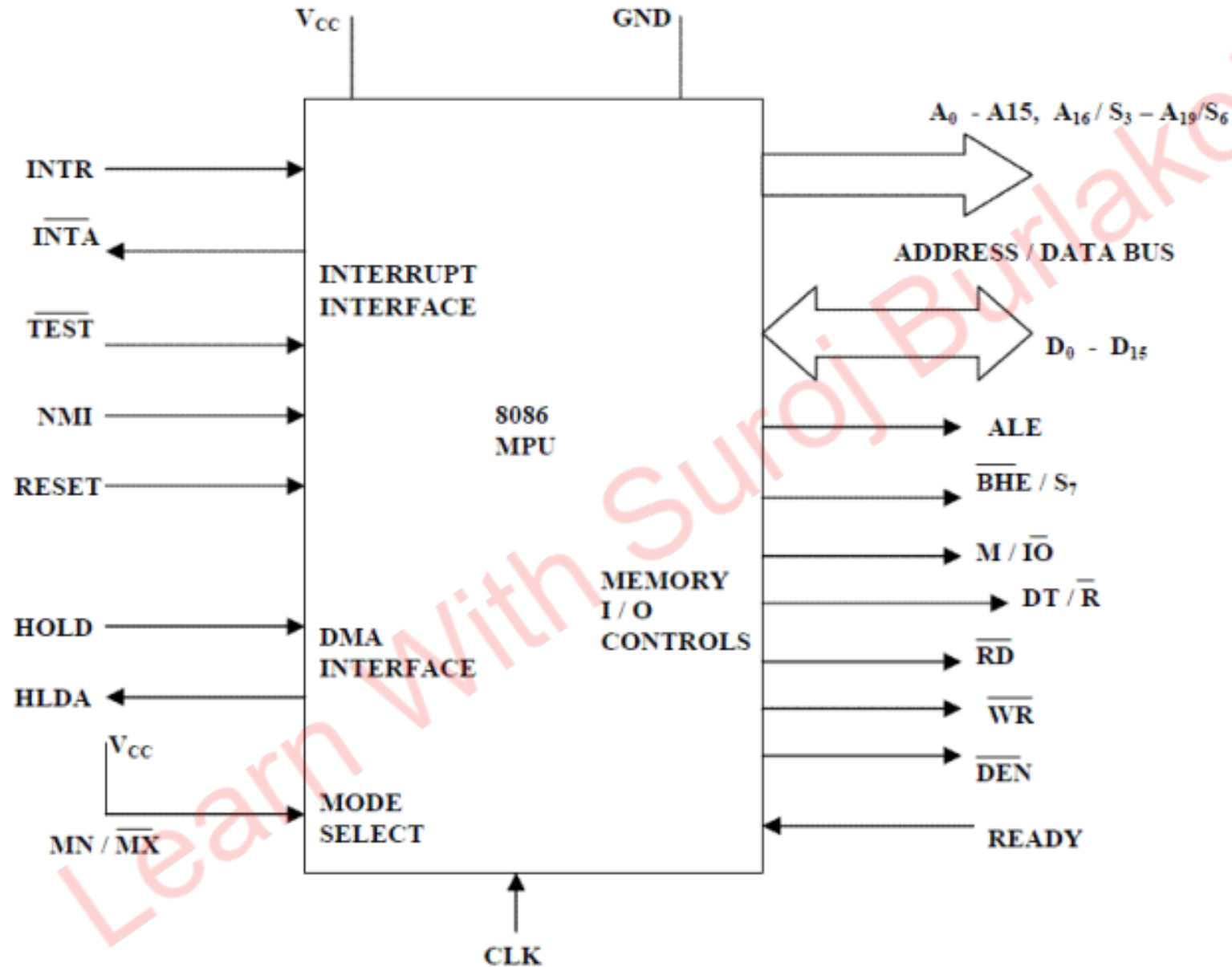
AH= no of screen columns **INT 10H**

BH = active video page

Pin Diagram of 8086



Signal Groups of 8086



The following signal descriptions are common for both modes.

- **AD15-AD0** : These are the time multiplexed memory I/O address and data lines.
- **A19/S6,A18/S5,A17/S4,A16/S3** : These are the time multiplexed address and status lines. The address bits are separated from the status bit using latches controlled by the ALE signal.
- **BHE/S7** : The bus high enable is used to indicate the transfer of data over the higher order (D15-D8) data bus.
- **RD – Read** : This signal on low indicates the peripheral that the processor is performing memory or I/O read operation.
- **READY** : This is the acknowledgement from the slow device or memory that they have completed the data transfer.
- **INTR-Interrupt Request** : This is to determine the availability of the request from external devices. If any interrupt request is pending, the processor enters the interrupt acknowledge cycle.
- **TEST** : This input is examined by a ‘WAIT’ instruction. If the TEST pin goes low, execution will continue, else the processor remains in an idle state.
- **CLK- Clock Input** : The clock input provides the basic timing for processor operation and bus control activity.

The following pin functions are for the minimum mode operation of 8086.

- **M/IO – Memory/IO :** When it is low, it indicates the CPU is having an I/O operation, and when it is high, it indicates that the CPU is having a memory operation.
- **INTA – Interrupt Acknowledge:** This signal is used for interrupt acknowledge i.e. when it goes low; the processor has accepted the interrupt.
- **ALE – Address Latch Enable:** This output signal indicates the availability of the valid address on the address/data lines.
- **DT/R – Data Transmit/Receive:** This output is used to decide the direction of data flow through the trans-receivers (bidirectional buffers). When the processor sends out data, this signal is high and when the processor is receiving data, this signal is low.
- **DEN – Data Enable:** This signal indicates the availability of valid data over the address/data lines. It is used to enable the trans-receivers (bidirectional buffers) to separate the data from the multiplexed address/data signal.
- **HOLD, HLDA- Acknowledge:** When the HOLD line goes high, it indicates to the processor that another master is requesting the bus access. The processor, after receiving the HOLD request, issues the hold acknowledge signal on.

The following pin functions are applicable for maximum mode operation of 8086.

- **S2, S1, S0 – Status Lines:** These are the status lines which reflect the type of operation, being carried out by the processor.

S2	S1	S0	Indication
0	0	0	Interrupt Acknowledge
0	0	1	Read I/O port
0	1	0	Write I/O port
0	1	1	Halt
1	0	0	Code Access
1	0	1	Read Memory
1	1	0	Write Memory
1	1	1	Passive

- **LOCK :** This output pin indicates that other system bus master will be prevented from gaining the system bus, while the LOCK signal is low.
- **RQ/GT0, RQ/GT1 – Request/Grant :** These pins are used by the other local bus master in maximum mode, to force the processor to release the local bus at the end of the processor current bus cycle.

Thank You

Learn With Guraj Burlakoti